

Library Management System

Project Report

Description

This project involves creating a multi-user digital library application to be designed for librarians and students/general users. The application facilitates e-book management. This digital library application offers a user-friendly platform for librarians to manage e-books and user access. Students and general users can benefit from a convenient way to borrow and read e-books through a centralized system.

Technologies used

Some of the key technologies that has been used are:

- **Flask:** Used for microframework for building the library management system website in python.
- **Flask-SQLAlchemy:** Used in simplifying database interaction in Flask applications.
- **Jinja2 templates:** Used with Flask to achieve a clear separation of concerns between the presentation layer (HTML, CSS) and the application logic (Python code).
- **Flask-RESTful:** Enabled building RESTful APIs using Flask.
- **Datetime:** Provided date and time objects and functionalities for storing time of return.
- **Requests:** Used for fetching data from external web pages.
- **Matplotlib:** Creates static, animated, and interactive visualizations in Python.

DB Schema Design

- **User:** Represents a user of the library system with attributes like id, user_id (unique identifier), firstName, password, and count_access (tracks the number of books a user can access). It has a one-to-many relationship with User_Book, meaning a user can have many book requests/issuances.
- **Book:** Represents a book in the library with attributes like Book_id, Book_Name, Author, Synopsis, Content, and section_id (foreign key referencing the Section table). It has a many-to-many relationship with User_Book, indicating a book can be requested/issued by multiple users. It also has a one-to-many relationship with User_Book through feedback, allowing users to provide feedback on a specific book instance.
- **Section:** Represents a section in the library with attributes like Section_id, Section_Title (unique identifier), Section_Date, and Section_Descript. It has a one-to-many relationship with Book, signifying that a section can contain multiple books.
- **User_Book:** This table acts as a join table between User and Book, storing information about book requests/issuances. It has attributes like user_book_id (primary key),

User_id (foreign key referencing User table), Book_id (foreign key referencing Book table), Section_id (foreign key referencing Section table), Status (indicates the status of the request - Pending, Issued, Rejected), expiration_period (stores the expiration date/time for issued books), and feedback (stores user feedback for a returned book).

Features

User Functionalities:

- Browse sections and available books on the home page.
- Filter search results based on books/authors/sections.
- Request books based on access limits.
- Choose the duration for borrowing (hours, days, weeks).
- See a list of their requested, issued, and rejected books.
- Return borrowed books and provide feedback.
- Read feedback from other users on returned books.
- Generate charts visualizing issued and rejected books per section.

Admin Functionalities:

- Add new sections to the library system.
- Add new books to the library system, categorized under sections.
- Edit/Delete existing sections only upon deleting the books present in it.
- Edit /Delete existing books.
- View a list of all pending, issued, and rejected book requests.
- Approve or reject book requests, updating user access count accordingly.
- Revoke access to issued books for both users.

Folder Architecture

The project is organized with distinct components.

- The Website folder contains the codes related to database models, routes, and templates.
- API endpoints and routes are found in api.py and views.py, handling user interactions and logic.
- Database models are defined in models.py using SQLAlchemy.
- HTML templates reside in the templates folder.

Vaijayanti Mallick
21F1004680
21f1004680@ds.study.iitm.ac.in